ASTER : MLIR-Based Assembly Tooling and Representations

Nicolas Vasilache (speaker), Fabian Mora Cordero, Kunwar Grover

AMD

Euro LLVM 2026 MLIR Workshop — April 13, 2026

Preliminaries

The Performance Control Gap

Current State of GPU Compilers

The Challenge

Modern ML is the most demanding perf ecosystem in history (>> HPC)

Today, achieving **peak performance** on modern AI accelerators requires:

- Direct control over low-level hardware features (MFMA instructions, swizzles, etc)
- Managing asynchronicity and dynamism (async, pipelining, multi-buffering)
- Expert-level understanding of instruction scheduling
- Control over register allocation and instruction issue ports

Other observe this too: DeepSeek @PTX, Gluon as Triton lacks perf, etc

The Reality

Traditional compiler stacks hide critical details behind opaque abstraction layers

The Dark Silicon Problem

Hardware Becomes More Complex and Heterogeneous

- **Coarser-grain primitives** exposed by hardware (bigger and more systolic, fancier n-D DMAs, more MPI-y)
- **Warp/wave specialization** requiring explicit management
- **Programming models** resembling MPI/MIMD-style parallelism
- **Low-level constraints**: instruction ports, wave scheduling

What This Means

The gap between "write once" and "peak performance" is growing: embrace it

AMD's Open Approach

A Unique Opportunity

AMD's **open hardware ISA documentation** enables:

- World-class assembly tooling built in the open
- Making AMDGPU ASM accessible to broader communities
- Higher-level tools with expert-level control

ASTER Vision

Order of magnitude better low-level tooling: same abstractions for HW designer, library experts, compiler writers, high-performance engineers

Reap the benefits of modern and future hardware, faster

ASTER Introduction

Assembly Tooling and Representations

What is ASTER?

ASTER is an open-source **MLIR-based tool** for programmable and highly-controllable assembly production on AMD GPUs

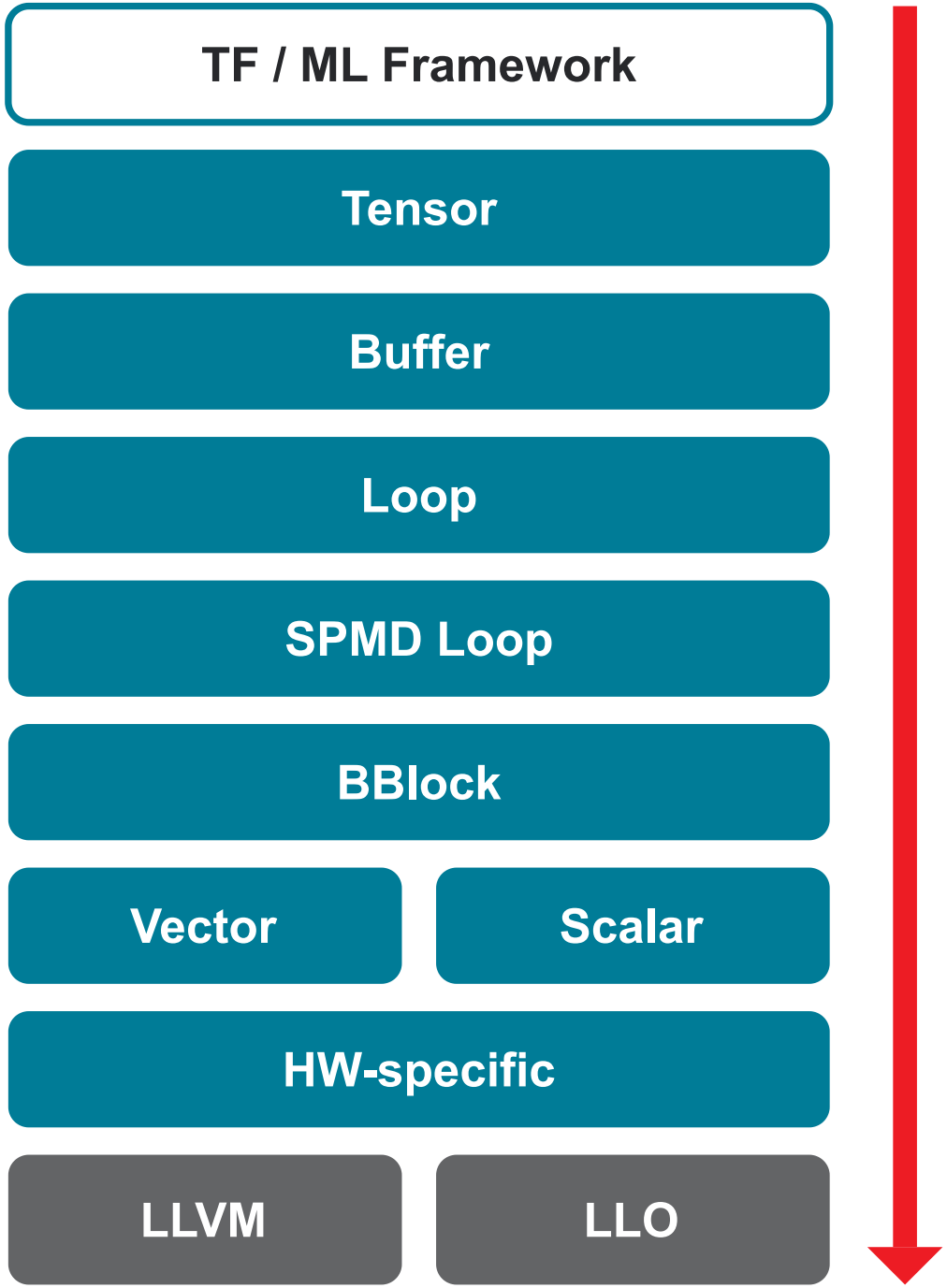
Core Mission

Allow 10x better low-level programming at peak

Bring modern compiler engineering principles to ASM generation

Make hard things possible and complex things manageable

Be a modern bottom-up complement to top-down structured code generation



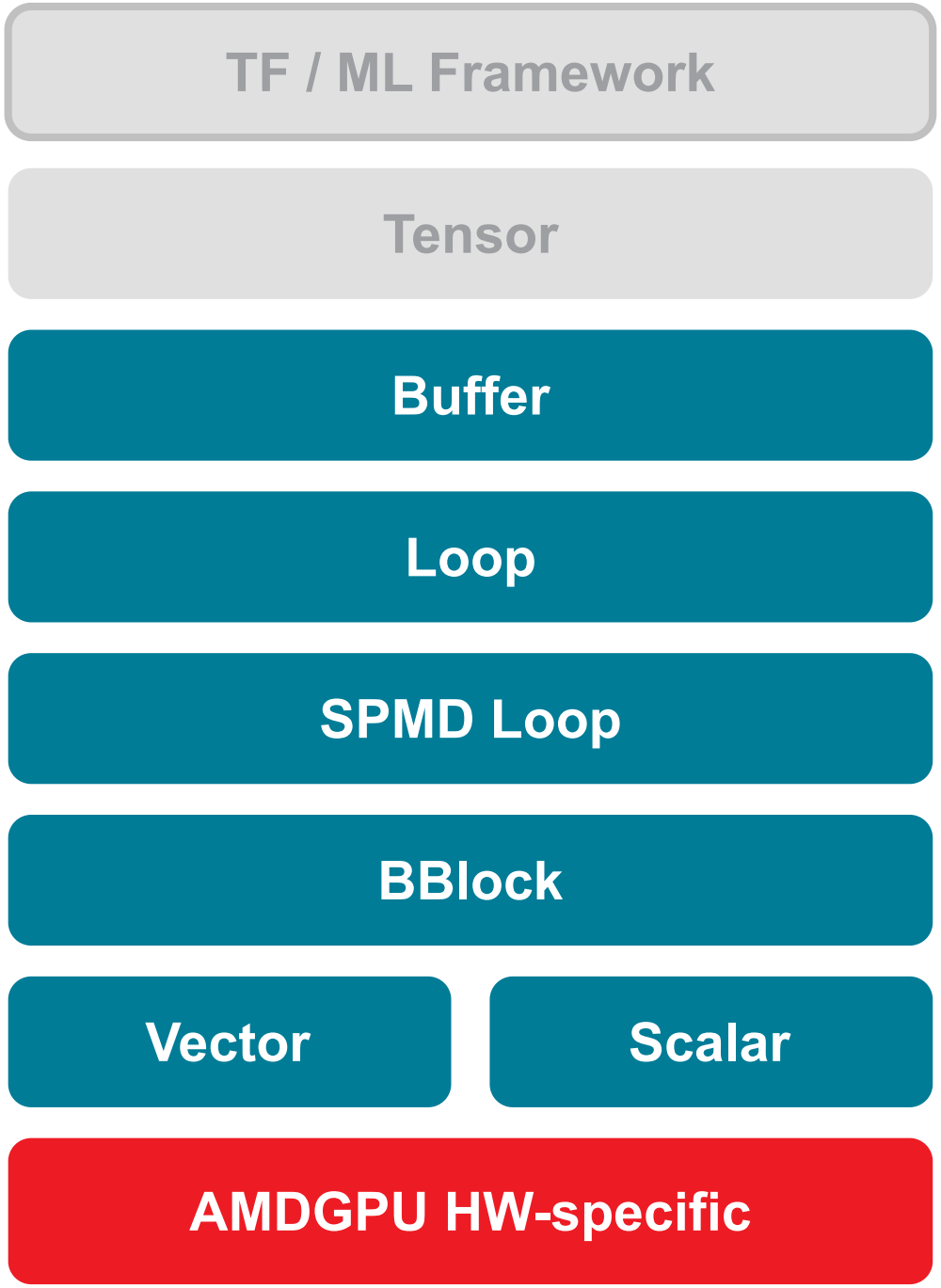
Top-Down Structured Code Generation: Decomposition-Driven

Transformations First
Expressiveness, composability, simplicity

Preserve and infer information at each level

IR remains closed under transformations
No leaking of lower-level details

Start from named, typed operations
Lower progressively toward hardware
Each level is a distinct compilation target



The Bottom-Up Code Generation: Composition-Driven

Performance First
Hardware constraints and measurements drive every decision

Factor out building blocks per hardware and across hardware generations

Separation of concerns -- bottom-up handles what top-down cannot observe:

- Register allocation and reuse
- Instruction scheduling and latency hiding

Reduce new problems to known problems

This is the role ASTER is designed to play

Three Core Principles

1. **Control First, Automatic Optimizations Second**
   - WYSIWYG: What You See Is What You Get
   - No unwanted compiler transformations by default
   - Opt-in composable automation

2. **Leverages Modern Compiler Infrastructure**
   - Clean MLIR dialect for AMDGPU instructions
   - SSA-based representation enables modern analyses
   - Type-safe, composable transformations

3. **Composable by Design**
   - Minimal, independent tool
   - Interacts with existing MLIR ecosystem
   - Partial register allocation and incremental compilation

Supported Targets

Target	ISA	Product Family
gfx940	CDNA3	MI300A
gfx942	CDNA3	MI300X
gfx950	CDNA4	MI350X
gfx1201	RDNA4	Radeon RX 9070

OSS HSACO generation requires up-to-date LLVM AMDGPU assembler

ASTER's IR and ASM translation works regardless of LLVM version

Examples

Simple to Complex

Example: Vector Addition

Building a kernel: `c[tid] = a[tid] + b[tid]`

```
kernel @kernel arguments <[...buffer args...]> {
  %v0 = alloca : !amdgcn.vgpr<0>  // thread ID
  %v1 = alloca : !amdgcn.vgpr<1>  // byte offset
  %v2 = alloca : !amdgcn.vgpr<2>  // a[tid], then result
  %v3 = alloca : !amdgcn.vgpr<3>  // b[tid]

  // Load buffer pointers via reusable function
  %a_ptr, %b_ptr, %c_ptr = func.call @load_3_ptrs(%kernarg) ...

  // Compute offset: tid * 4
  vop2 v_lshlrev_b32_e32 outs %v1 ins %c2, %v0

  // Load a[tid] and b[tid]
  amdgcn.load global_load_dword dest %v2 addr %a_ptr offset d(%v1) + c(%c0)
  amdgcn.load global_load_dword dest %v3 addr %b_ptr offset d(%v1) + c(%c0)
  amdgcn.sopp.s_waitcnt #amdgcn.inst<s_waitcnt> vmcnt = 0

  // Add and store
  vop2 v_add_u32 outs %v2 ins %v2, %v3
  amdgcn.store global_store_dword data %v2 addr %c_ptr offset d(%v1) + c(%c0)
  end_kernel
}
```

Key: Explicit registers, reusable functions, direct control, type safety

MLIR to Assembly: Direct Translation

```
%v0 = alloca : !amdgcn.vgpr<0>
%v1 = alloca : !amdgcn.vgpr<1>
%v2 = alloca : !amdgcn.vgpr<2>

%c32 = arith.constant 32 : i32
amdgcn.vop1.vop1 <v_mov_b32_e32> %v1, %c32

%c10 = arith.constant 10 : i32
amdgcn.vop1.vop1 <v_mov_b32_e32> %v2, %c10

// v0 = v1 + v2 = 32 + 10 = 42
vop2 v_add_u32 outs %v0 ins %v1, %v2
```

Generates:

```
v_mov_b32_e32 v1, 32
v_mov_b32_e32 v2, 10
v_add_u32 v0, v1, v2
```

Python Integration

Fast iteration: edit → compile → execute in seconds

```python
from aster import ir
from aster.dialects.kernel_builder import KernelBuilder, AccessKind

def build_add42():
    """Out[tid] = in[tid] + 42"""
    b = KernelBuilder("add42", "kernel", target="gfx942", isa="cdna3")

    b.add_ptr_arg(AccessKind.ReadOnly)   # input
    b.add_ptr_arg(AccessKind.WriteOnly)  # output
    [in_ptr, out_ptr] = b.load_args()

    tid = b.thread_id("x")
    byte_off = b.affine_apply(ir.AffineExpr.get_dim(0) * 4, [tid])

    in_addr = b.global_addr(in_ptr, byte_off)
    val, tok = b.global_load_dword(in_addr)
    b.wait_deps(tok)

    result = b.v_add_u32(b.constant_i32(42), val)

    out_addr = b.global_addr(out_ptr, byte_off)
    tok = b.global_store_dword(result, out_addr)
    b.wait_deps(tok)

    return b.build()
```

Reuse MLIR upstream infra, transformations and lowerings where it makes sense

Simple numpy integration: no framework dependency, lightweight, just works

ASTER Internals

MLIR Dialect Design

Semantic Grouping

One MLIR operation represents a **family** of similar instructions
Example: `amdgcn.load` covers `global_load_*`, `buffer_load_*`, etc.

Multi-level representation

- Value SSA semantics: `!amdgcn.vgpr_range<[? + 4]>`
- Side-effect semantics: `!amdgcn.vgpr_range<[0 : 4]>`

Destination-Passing Style (DPS)

Explicit destination operands, precise hints to register allocator

```
%r, %t = amdgcn.load ds_read_b32 dest %d addr %a offset c(%off) : dps(!vgpr) ins(!vgpr, i32) -> !amdgcn.read_token<shared>
```

AMDGCN Backend: Key Compiler Passes

ASTER includes composable compiler passes for assembly production

Pass	Description
Pipelining	Significantly stepped up `scf.for` pipeliner with futures and "free" multi-buffering
LDS Allocation	Manage shared memory allocation and deallocation
Bufferization	Inspired by upstream MLIR bufferization patterns
Register Allocation	SSA-based interference analysis with partial user control
Wait Insertion	Automatic `s_waitcnt` insertion for memory dependencies
Hazard Handling	Detect and resolve hardware hazards and introduce nops

Register Allocation: Incremental & Partial

ASTER allows **mixing explicit and automatic** register allocation

Example: Critical Path Control

```
// User specifies critical registers for MFMA inputs/outputs
%v8 = alloca : !amdgcn.vgpr<8>
%v9 = alloca : !amdgcn.vgpr<9>
%v10 = alloca : !amdgcn.vgpr<10>
%v11 = alloca : !amdgcn.vgpr<11>

// Compiler auto-allocates temporaries
%temp1 = amdgcn.alloca : !amdgcn.vgpr  // → v12 (auto)
%temp2 = amdgcn.alloca : !amdgcn.vgpr  // → v13 (auto)

// Load data into auto-allocated registers
%data = amdgcn.load global_load_dword dest %temp1 addr %ptr

// Use in MFMA with explicit registers
%c_range = amdgcn.make_register_range %v8 : !amdgcn.vgpr_range<[8 + 4]>
%result = amdgcn.vop3p.vop3p_mai <v_mfma_f32_16x16x16_f16>
  %c_range, %a_data, %b_data, %c_range
```

Memory Tokens: Fine-grained Dependency Tracking

ASTER uses **explicit tokens** to track memory dependencies

Gives a very compelling Async / Future-based programming model

```
// Load returns data + token
%c0 = arith.constant 0 : i32
%load_dest = amdgcn.alloca : !amdgcn.vgpr
%data, %read_tok = amdgcn.load global_load_dword dest %load_dest addr %ptr
  offset d(%voffset) + c(%c0)
  : dps(!amdgcn.vgpr) ins(!amdgcn.sgpr<[? + 2]>, !amdgcn.vgpr, i32)
    -> !amdgcn.read_token<flat>

// Token types encode memory space
// !amdgcn.read_token<flat>   - global memory reads
// !amdgcn.read_token<shared> - LDS memory reads
// !amdgcn.write_token<flat>  - global memory writes
// !amdgcn.write_token<shared> - LDS memory writes
```

Why Tokens?

- **Fine-grained**: Compiler tracks which operations depend on which memory ops
- **Explicit ordering**: Users control when synchronization happens
- **Composability**: Token types distinguish memory spaces automatically
- **Retargetable**: Async/Future lowers to different HW models of concurrency

LSIR: Load-Store Intermediate Representation

LSIR is an intermediate dialect bridging high-level IR and AMDGCN assembly

What is LSIR?

A **load-store register architecture** IR providing:

- **Destination-passing style (DPS)** for all operations
- **Explicit memory operations**: `lsir.load` and `lsir.store` with tokens
- **Arithmetic/logical ops**: `lsir.addi`, `lsir.andi`, `lsir.shli`, etc.
- **Control flow**: `lsir.cmpi`, `lsir.select`

Why LSIR?

- **Lowering target**: Higher-level dialects (e.g., `arith`) lower to LSIR
- **Register-aware**: Operations work with register types (`!amdgcn.vgpr`, `!amdgcn.sgpr`)
- **Token-based dependencies**: Memory operations return tokens for synchronization
- **Clean abstraction**: Simplifies lowering to AMDGCN assembly

LSIR Example: Arithmetic Lowering

High-level arith dialect lowers to LSIR

```
// High-level arith operation
func.func @test_maxui(%a: i32, %b: i32) -> i32 {
  %r = arith.maxui %a, %b : i32
  return %r : i32
}
```

Lowers to LSIR with DPS:

```
func.func @test_maxui(%a: !amdgcn.vgpr, %b: !amdgcn.vgpr) -> !amdgcn.vgpr {
  %dst = lsir.alloca : !amdgcn.vgpr
  %result = lsir.maxui i32 %dst, %a, %b
    : !amdgcn.vgpr, !amdgcn.vgpr, !amdgcn.vgpr
  return %result : !amdgcn.vgpr
}
```

Layout Dialect: Structured Address Computation

Layout dialect provides high-level abstractions for mapping logical coordinates to physical memory offsets

What is the Layout Dialect?

- **Strided layouts**: Define shape and stride patterns with `#layout.strided_layout<[shape] : [stride]>`
- **Nested layouts**: Support hierarchical tiling patterns
- **Swizzle operations**: XOR-based patterns for LDS bank conflict avoidance
- **Clean lowering**: Lowers to `affine` dialect operations

Why Layouts?

- **Separation of concerns**: Address computation logic separate from memory operations
- **Reusable patterns**: Common layouts expressed once, used everywhere
- **Optimization**: Compiler can optimize layout arithmetic
- **Hardware-aware**: Swizzle patterns match hardware memory banking

Layout Example: Row-Major vs Column-Major

```
// Row-major 4x8 array (first mode stride 8, second mode stride 1)
func.func @rowmajor(%c: index) -> index {
  %off = layout.linearize %c, #layout.strided_layout<[4, 8] : [8, 1]>
  return %off : index
}

// Column-major 4x8 array (first mode stride 1, second mode stride 4)
func.func @colmajor(%c: index) -> index {
  %off = layout.linearize %c, #layout.strided_layout<[4, 8] : [1, 4]>
  return %off : index
}
```

Lowering to Affine

```
// Row-major lowers to:
%c0, %c1 = affine.delinearize_index %c into (4, 8)
%off = affine.linearize_index_by_strides[%c0, %c1] by (8, 1) : index

// Column-major lowers to:
%c0, %c1 = affine.delinearize_index %c into (4, 8)
%off = affine.linearize_index_by_strides[%c0, %c1] by (1, 4) : index
```

Execution Infra: Tuner Heuristics and Search

Dozen low-level knobs: library generation as a composition of compiler transformations

Concurrent multi-GPU compile, run, measure loop (10 - 100 evaluations per second per node)

Performance Control

Performance Control

Perfect Control Mode

Perfect control over generated assembly lets one:

- deeply understand every piece of hardware by pico-benchmarking
- verify, test, debug the hardware
- in a programmable fashion without compiler interference (e.g. DCE)

Pico-benchmark: 10 Sequential v_add_u32

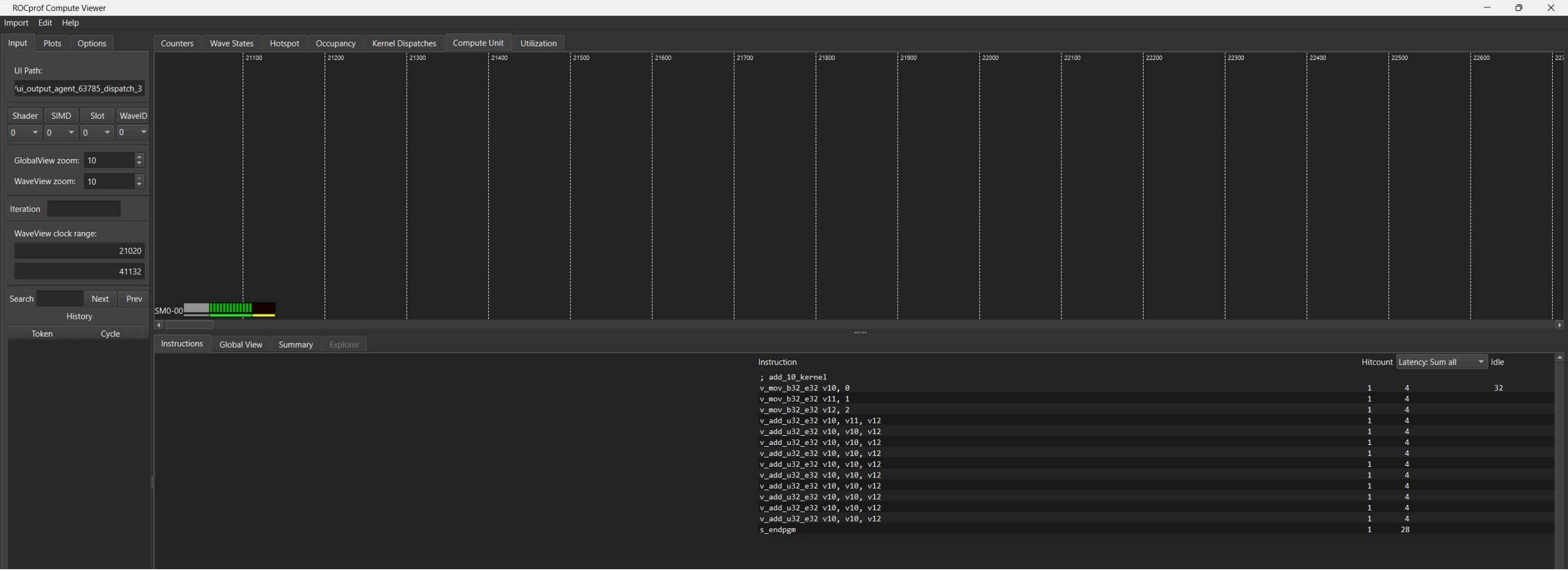
`demo/add_10.mlir`: 10 dependent adds, empty pass pipeline

WYSIWYG: manually scheduled, manually register-allocated

No DCE, no reordering, no compiler interference

`rocprofv3` trace shows each instruction issues exactly as written

Pico-benchmark: 10 Sequential v_add_u32



Pico-benchmark: MFMA Issue Limits

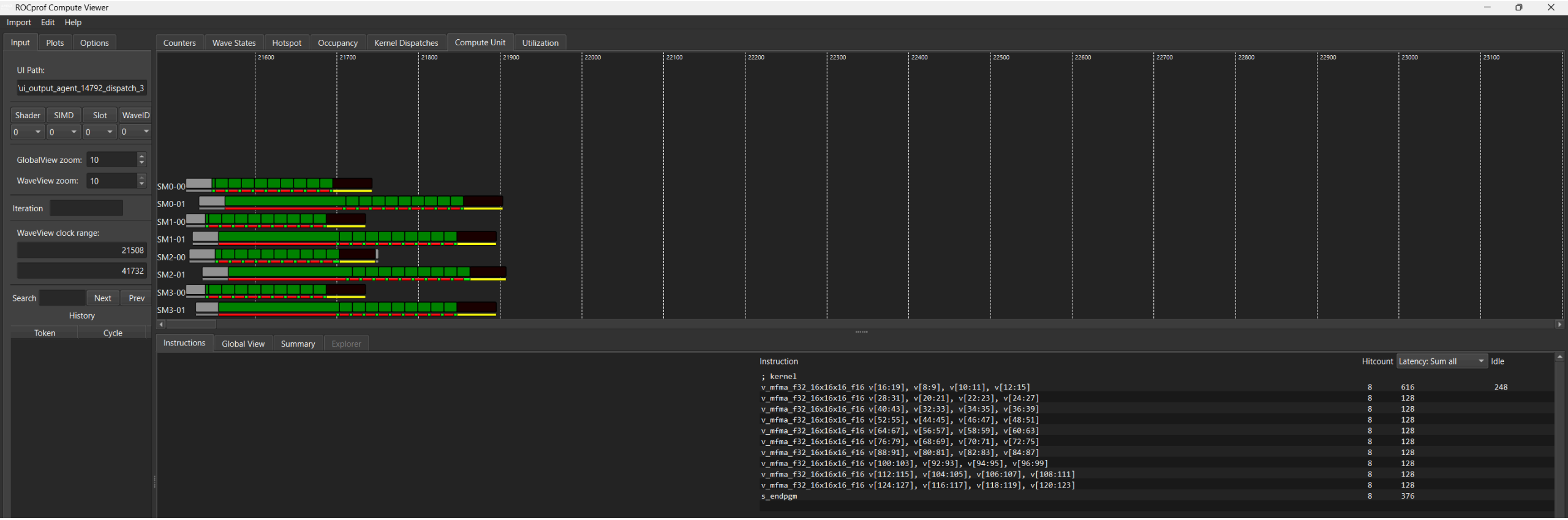
`demo/generate_mfma_10.py`: 10 independent MFMAs per wave,
8 waves per CU, 304 CUs on MI300X

Observations from a single trace:

- 1st MFMA issues in **~4 cycles**
- Subsequent MFMAs wait **16 cycles** -- no issue buffering on MATRIX unit
- 2nd co-located wave serialized after 1st completes

Simple kernels, deep hardware insight

Pico-benchmark: MFMA Issue Limits



Gradual Automation Mode

(Very) preliminary numbers, TFLOP/s, higher is better.

`C = A @ B^T`, `f16 x f16 -> f32`, MI300x

M x N x K	IREE	Triton	ASTER	RocBLAS	HIPBlasLt	AITER	vs Best Compiled	vs Best
4096 x 4096 x 4096	324	561	**549**	481	570	517	97.7%	96.3%
2432 x 12288 x 4096	363	--	**586**	426	569	551	137%	103%
4864 x 9216 x 8192	407	--	**575**	450	604	603	127%	95.1%
4864 x 18432 x 4096	368	--	**556**	492	575	554	113%	96.7%

- **Within 4-5% of hand-tuned vendor libraries** (HIPBlasLt, AITER) -- still missing low-level scheduling
- **100% MLIR**: no handwritten assembly, no opaque library calls, fully controllable

Conclusion

What ASTER Enables

- **Fast iteration**: Fast time to insight: authoring, tuning and compile times
- **Composability**: Mix hand-optimized kernels with generated code
- **Hardware exploration**: Deep understanding through measurement, even debugging (e.g. NOP hazards)
- **Interoperability**: MLIR ecosystem means tools compose
- **Future-prrof**: We expect much faster time to insight on future HW (mi4xx and beyond)

What's Next?

Upcoming Infrastructure

- Raising existing ASM to MLIR for commonalization, editing and tuning
- Lowerings from higher-level MLIR codegen to ASTER
- More reusable MLIR nanokernels for unsurprisingly good performance
- More target and feature support
- Perf, perf, perf: squeeze out every cycle
- Integration with ML compilation flows (IREE, Triton, ...)
- Non-GPU: AIE, DSM, CPUs etc

Thank You

Questions?

Repository: https://github.com/iree-org/aster

Contact: Nicolas Vasilache, Fabian Mora Cordero, Kunwar Grover (AMD)

Enabling world-class assembly tooling in the open